Assignment-3

COMPARISON OF SIMILARITY METRICS FOR TEXTURE IMAGE RETRIEVAL.

Submitted By:

Sai Charan Kandepi

Matriculation no: 12501718

Guided By: Prof. DR. Marcus Barkowsky

Problem Description - The Challenge of Image Retrieval

- Rapid growth of digital image databases demands efficient retrieval methods.
- Traditional keyword-based retrieval is time-consuming and inadequate for visual content.
- Content-Based Image Retrieval (CBIR) focuses on visual features like texture.
- While texture features can be extracted effectively,
  - Similarity measurement between images is still a major challenge.
- Most systems use standard distance metrics (e.g., Euclidean) without assessing their impact on retrieval accuracy.

Paper's Proposed Solution

Feature Extraction Using Gabor Wavelets:

Applied a Gabor filter bank with:

- 4 scales × 6 orientations = 24 filters

For each filtered image:

- Calculated mean and standard deviation
- Resulting in a 48-dimensional feature vector per image.

Similarity Metrics Compared (9 Total):

- Euclidean (L2)
- Manhattan (L1)
- Mahalanobis
- Chebyshev
- Weighted-Mean-Variance (WMV)
- Canberra
- Bray-Curtis
- Squared Chord
- Squared Chi-Squared

Texture Retrieval Procedure (Original Paper)

1. Database Preparation
- Used 116 textures (Brodatz + USC + 1 artificial)
- Each 512×512 image split into sixteen 128×128 patches → 1856 images

2. Feature Extraction
- Applied Gabor filters (4 scales × 6 orientations = 24)
- Computed mean & standard deviation for each → 48D feature vector

3. Feature Database Creation

Stored 48D vectors of all 1856 images in a feature database

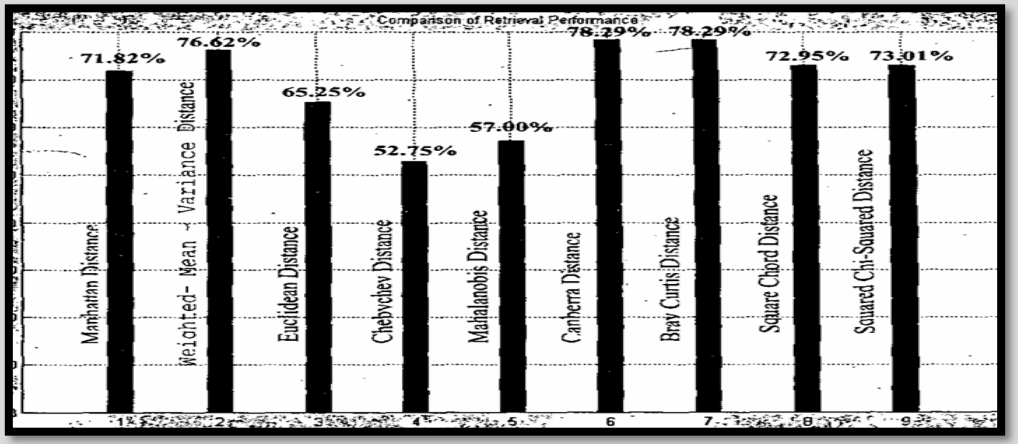
4. Query Processing

Query image also processed into a 48D vector (same pipeline)

5. Similarity Measurement
- Compared query vs. database using 9 distance metrics(e.g., Euclidean, Mahalanobis, Canberra, Bray-Curtis)
- Explored which metric yields best retrieval accuracy

6. Ranking & Retrieval
- Ranked database images by distance (smaller = more similar)
- Top matches retrieved (ideally all 16 from the same original texture)

7. Experimental Evaluation
- Implemented in MATLAB
- Found **Canberra and Bray-Curtis** gave highest retrieval accuracy



Gabor Wavelets

2D Gabor Function

- Gabor function form a complete but non-orthogonal basis set.

$$g(x,y)=\left(\frac{1}{2\pi\sigma_x\sigma_y}\right)\exp\left[-\frac{1}{2}\left[\frac{x^2}{\sigma^2{}_x}+\frac{y^2}{\sigma^2{}_y}\right]+2\pi j W_x\right]$$

where $\sigma_u = 1/2\pi\sigma_x$ and $\sigma_v = 1/2\pi\sigma_y$.

What is a Gabor Filter Bank?

- Instead of a single filter, a "Gabor filter bank" (also referred to as a "self-similar filter dictionary") is used. This bank consists of multiple Gabor filters, each tuned to different frequencies and orientations.
- This "self-similar filter dictionary" is generated by applying appropriate dilation (for scales) and rotations (for orientations) to a single "mother Gabor wavelet," g(x,y).
  - Dilation: A scale factor, a^-m, is applied to control the filter's size and frequency response, ensuring that the energy of the filter is independent of its scale m.
  - Rotation: Rotations are defined by an angle θ = nπ/K, where K is the total number of orientations

$$g_{mn}(x,y)=a^{-m}\,G(x,y), \quad a>1, \quad m,n = \text{intege}$$
$$x'=a^{-m}(x\cos\theta+y\sin\theta),$$
$$y'=a^{-m}(-x\sin\theta+y\cos\theta), \qquad (:$$

SIMILARITY MEASURES

- The distance metric can be termed as similarity measure, which is the key component in content-based image retrieval. It is important to explore different similarity measures to find the best distance metric for content based image retrieval.

1. The Euclidean or L2 metric is

$$d_E(x,y)=\sqrt{\sum_{i=1}^{d}(x_i-y_i)^2}$$

2. The Manhattan or L1 Metric is

$$d_M(x,y)=\sum_{i=1}^{d}|x_i-y_i|$$

3. Weighted- Mean-Variance

$$d_{Wmv}(x,y)=\sum_m\sum_n d_{mn}(x,y),$$

$$\text{where } d_{mn}(x,y)=\left|\frac{\mu^x{}_{mn}-\mu^y{}_{mn}}{\sigma(\mu_{mn})}\right|+\left|\frac{\sigma^x{}_{mn}-\sigma^y{}_{mn}}{\sigma(\mu_{mn})}\right|$$

4. Mahalanobis Metric

$$d_{Mah}(x,y)=\sqrt{(x-y)'\,Cov(x)^{-1}(x-y)}$$

5. Canberra Metric

$$d_c(x,y)=\sum_{i=1}^{d}\frac{|x_i-y_i|}{|x_i|+|y_i|}$$

6. Bray Curtis metric

$$d_{BC}(x,y)=\sum_{i=1}^{d}\frac{|x_i-y_i|}{x_i+y_i}$$

7 & 8. Squared Chord and Square Chi-Squared distance

$$d_{sc}(x,y)=\sum_{i=1}^{d}\left(\sqrt{x_i}-\sqrt{y_i}\right)^2$$

$$d_{chi}(x,y)=\sum_{i=1}^{d}\frac{(x_i-y_i)^2}{x_i+y_i}$$

9. Chebychev

$$D_{Cheb}(x,y)=\max_i(|x_i-y_i|)$$

Code Implementation

1. **Image Database Creation and Preprocessing**

```python
import os
import cv2
import numpy as np
import matplotlib.pyplot as plt
import math
import time
# === Configuration ===
INPUT_DIR = "C:/Users/owner/Desktop/Real_Brodatz"
OUTPUT_DIR = "C:/Users/owner/Desktop/sub img"
TARGET_IMG_SIZE = 512
SUB_IMG_SIZE = 128


# === STEP 1: Resize and Subdivide Images ===
def resize_and_split_image(img_path, save_dir, target_size=512, sub_size=128):
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        print(f"Skipping {img_path}: Unable to read.")
        return

    img = cv2.resize(img, (target_size, target_size), interpolation=cv2.INTER_AREA)
    base_name = os.path.splitext(os.path.basename(img_path))[0]
    idx = 0


    for i in range(0, target_size, sub_size):
        for j in range(0, target_size, sub_size):
            sub_img = img[i:i+sub_size, j:j+sub_size]
            sub_img_name = f"{base_name}_sub_{idx:02d}.png"
            sub_img_path = os.path.join(save_dir, sub_img_name)
            cv2.imwrite(sub_img_path, sub_img)
            idx += 1


def process_all_images(input_dir, output_dir):
    image_files = [f for f in os.listdir(input_dir)
                   if os.path.isfile(os.path.join(input_dir, f))]
    total = 0
    for file_name in image_files:
        img_path = os.path.join(input_dir, file_name)
        resize_and_split_image(img_path, output_dir)
        total += 16
    print(f"Finished: Generated {total} sub-images from {len(image_files)} images.")
```

2. **Texture Feature Extraction: Gabor Wavelets**

```python
# === Parameters as per paper ===
sigma_x = 2.5
sigma_y = 2.5
W_x_list = [0.05, 0.1, 0.2, 0.4]  # Frequency band values
orientations_deg = [0, 30, 60, 90, 120, 150]
kernel_size = 31
FEATURE_SAVE_PATH = "C:/Users/owner/Desktop/gabor_features.npy"

# === STEP 2: Gabor Filter Definition with Scaling ===
def gabor_function_from_paper(x, y, sigma_x, sigma_y, W_x, theta, a_power_m_factor):
    # Use fixed sigma_x and sigma_y (mother wavelet spread) passed in; do NOT recalc here

    # Rotate coordinates
    x_rot = x * np.cos(theta) + y * np.sin(theta)
    y_rot = -x * np.sin(theta) + y * np.cos(theta)

    # Gaussian envelope (no scaling of coordinates)
    exponent = -0.5 * ((x_rot**2) / (sigma_x**2) + (y_rot**2) / (sigma_y**2))

    # Gabor function with cosine modulation
    gabor = (1 / (2 * np.pi * sigma_x * sigma_y)) * np.exp(exponent) * np.cos(2 * np.pi * W_x * x_rot)

    # Apply energy normalization factor as an outer multiplier
    gabor *= a_power_m_factor

    return gabor
```

```python
def generate_gabor_kernel(sigma_x, sigma_y, W_x, theta_deg, size, a_power_m_factor):
    theta = np.deg2rad(theta_deg)
    kernel = np.zeros((size, size), dtype=np.float32)
    half = size // 2
    for i in range(-half, half + 1):
        for j in range(-half, half + 1):
            kernel[i + half, j + half] = gabor_function_from_paper(i, j, sigma_x, sigma_y, W_x, theta, a_power_m_factor)
    return kernel
```

```python
# === STEP 3: Convolution and Feature Extraction ===
def convolve_image(img, kernel):
    h, w = img.shape
    kh, kw = kernel.shape
    pad_h, pad_w = kh // 2, kw // 2
    padded = np.pad(img, ((pad_h, pad_h), (pad_w, pad_w)), mode='reflect')
    out = np.zeros_like(img, dtype=np.float32)

    for i in range(h):
        for j in range(w):
            region = padded[i:i+kh, j:j+kw]
            out[i, j] = np.sum(region * kernel)
    return out


def extract_features(img, kernel_bank):
    features = []
    for kernel in kernel_bank:
        filtered = convolve_image(img, kernel)
        features.append(np.mean(filtered))
        features.append(np.std(filtered))
    return np.array(features, dtype=np.float32)


# === STEP 4: Main Pipeline ===
def main():
    # Step 1: Preprocess images
    process_all_images(INPUT_DIR, OUTPUT_DIR)

    # Step 2: Generate Gabor kernel bank (24 kernels)
    kernel_bank = []
    for m, W_x in enumerate(W_x_list):  # m = 0 to 3
        a_power_m_factor = (2) ** (-m)  # a = 2, so a^-m
        for theta in orientations_deg:
            kernel = generate_gabor_kernel(sigma_x, sigma_y, W_x, theta, kernel_size, a_power_m_factor)
            kernel_bank.append(kernel)

    # Step 3: Extract features from each sub-image
    features = []
    file_list = sorted([f for f in os.listdir(OUTPUT_DIR) if f.endswith(('.png', '.jpg'))])
    for fname in file_list:
        path = os.path.join(OUTPUT_DIR, fname)
        img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
        if img is None:
            print(f"Skipping {fname}")
            continue
        vec = extract_features(img, kernel_bank)
        features.append(vec)
```

3. Feature Database Creation

```python
# Step 4: Save features
feature_matrix = np.array(features)
np.save(FEATURE_SAVE_PATH, feature_matrix)
print(f"Done! Extracted features for {len(features)} images. Saved at: {FEATURE_SAVE_PATH}")
```

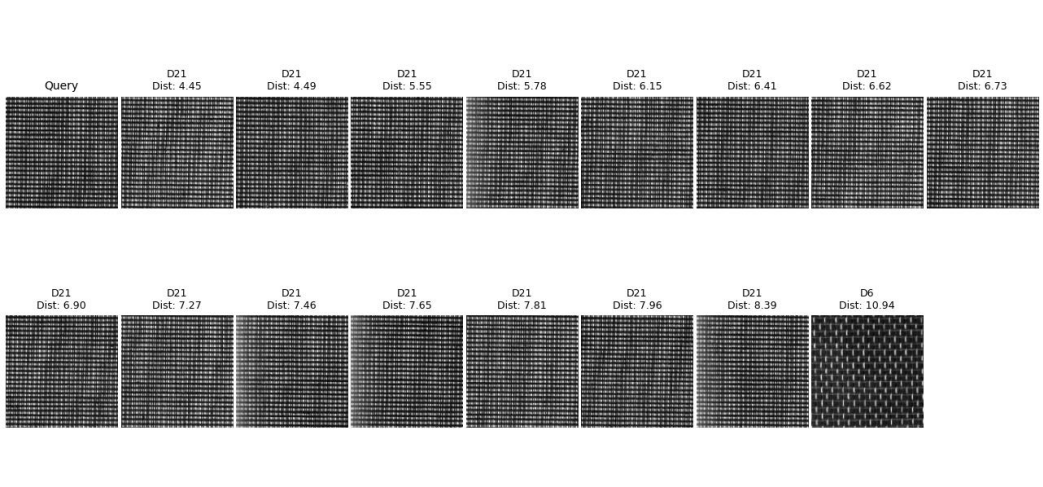
4. Similarity Measures/ Distance Metrics

```python
# === Distance Metrics ===
def manhattan_distance(x, y):
    return np.sum(np.abs(x - y))

def weighted_mean_variance_distance(x, y, std):
    return np.sum(np.abs(x - y) / std)

def euclidean_distance(x, y):
    return np.sqrt(np.sum((x - y) ** 2))

def chebyshev_distance(x, y):
    return np.max(np.abs(x - y))

def mahalanobis_distance(x, y, inv_cov):
    diff = x - y
    val = np.dot(diff, np.dot(inv_cov, diff))
    val = max(val, 0)
    return np.sqrt(val)


def canberra_distance(x, y):
    x = np.asarray(x)
    y = np.asarray(y)
    return np.sum(np.abs(x - y) / (np.abs(x) + np.abs(y)))

def bray_curtis_distance(x, y):
    x = np.asarray(x)
    y = np.asarray(y)
    return np.sum(np.abs(x - y)) / np.sum(x + y)

def squared_chord_distance(x, y):
    x = np.asarray(x)
    y = np.asarray(y)
    return np.sum((np.sqrt(x) - np.sqrt(y)) ** 2)


def squared_chi_squared_distance(x, y):
    num = (x - y) ** 2
    denom = x + y
    return np.sum(num / denom)
```

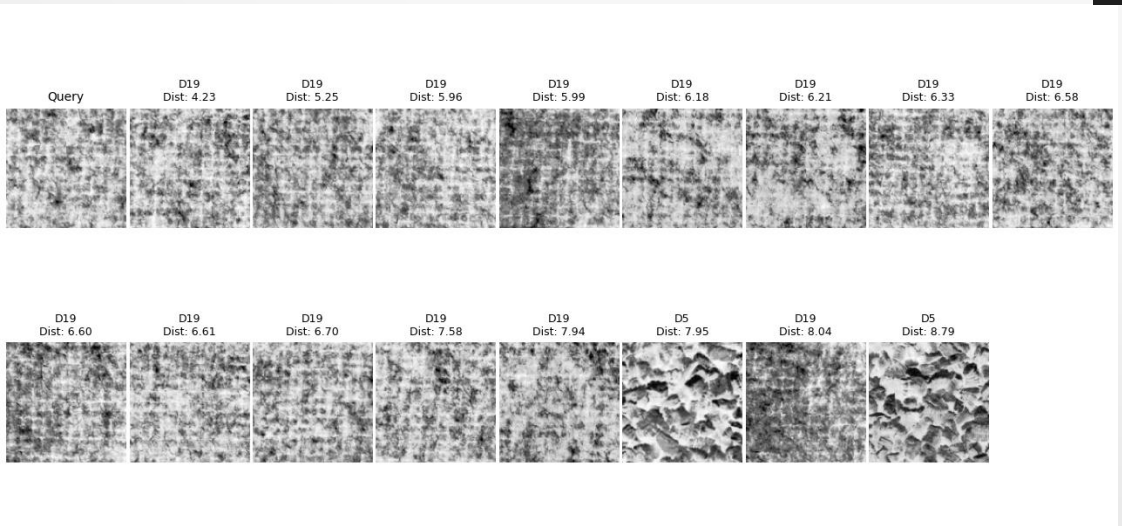
```python
def evaluate_all_queries(top_N=16):

    start_time = time.time()

    database_features = np.load(FEATURE_SAVE_PATH)
    file_list = sorted([f for f in os.listdir(OUTPUT_DIR) if f.endswith(('.png', '.jpg'))])
    num_images = len(file_list)

    std = np.std(database_features, axis=0)
    std[std == 0] = 1e-5

    cov = np.cov(database_features.T)
    inv_cov = np.linalg.pinv(cov)

    total_relevant_retrieved = {metric: 0 for metric in [
        "Manhattan", "Weighted-Mean-Variance", "Euclidean", "Chebyshev", "Mahalanobis",
        "Canberra", "Bray-Curtis", "Squared-Chord", "Squared-Chi-Squared"
    ]}
```

```python
def get_parent_class(fname):
    return fname.split('_sub_')[0]

for query_index in range(num_images):
    query_vec = database_features[query_index]
    query_name = file_list[query_index]
    query_class = get_parent_class(query_name)

    distances = {
        "Manhattan": [],
        "Weighted-Mean-Variance": [],
        "Euclidean": [],
        "Chebyshev": [],
        "Mahalanobis": [],
        "Canberra": [],
        "Bray-Curtis": [],
        "Squared-Chord": [],
        "Squared-Chi-Squared": []
    }

    for i, db_vec in enumerate(database_features):
        if i == query_index:
            continue
        db_name = file_list[i]
        distances["Manhattan"].append((db_name, manhattan_distance(query_vec, db_vec)))
        distances["Weighted-Mean-Variance"].append((db_name, weighted_mean_variance_distance(query_vec, db_vec, std)))
        distances["Euclidean"].append((db_name, euclidean_distance(query_vec, db_vec)))
        distances["Chebyshev"].append((db_name, chebyshev_distance(query_vec, db_vec)))
        distances["Mahalanobis"].append((db_name, mahalanobis_distance(query_vec, db_vec, inv_cov)))
        distances["Canberra"].append((db_name, canberra_distance(query_vec, db_vec)))
        distances["Bray-Curtis"].append((db_name, bray_curtis_distance(query_vec, db_vec)))
        distances["Squared-Chord"].append((db_name, squared_chord_distance(query_vec, db_vec)))
        distances["Squared-Chi-Squared"].append((db_name, squared_chi_squared_distance(query_vec, db_vec)))
```

Results

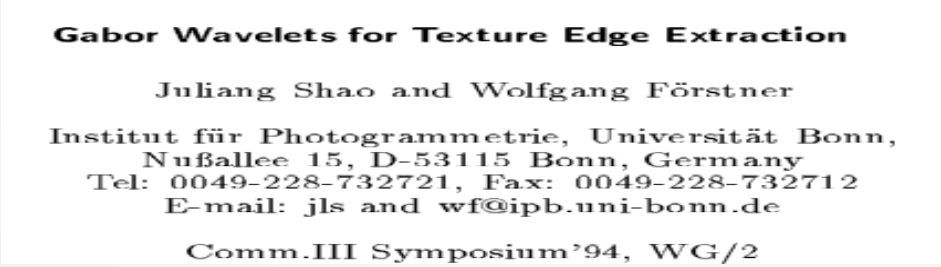



```
Average Retrieval Accuracy (Precision@Top-16):
  Manhattan: 45.06% (Time: 1.9786s)
  Weighted-Mean-Variance: 70.85% (Time: 1.9007s)
  Euclidean: 42.10% (Time: 2.0009s)
  Chebyshev: 41.54% (Time: 1.7402s)
  Mahalanobis: 34.18% (Time: 2.5438s)
  Canberra: 69.35% (Time: 2.9351s)
  Bray-Curtis: 45.17% (Time: 3.3038s)
  Squared-Chord: 3.45% (Time: 3.9733s)
  Squared-Chi-Squared: 55.83% (Time: 2.0629s)

Total Retrieval Time for All Queries: 24.0203 seconds
```



References

1. Aswini. K. Samantaray, Amol D. Rahulkar, “Comparison of Similarity Measurement Metrics on Medical Image Data”, IEEE – 45670, Available: https://ieeexplore.ieee.org/document/8944781
2. Juliang Shao and Wolfgang Förstner, “Gabor Wavelets for Texture Edge Detection“.



Gabor Wavelets for Texture Edge Extraction

Juliang Shao and Wolfgang Förstner

Institut für Photogrammetrie, Universität Bonn,
Nußallee 15, D-53115 Bonn, Germany
Tel: 0049-228-732721, Fax: 0049-228-732712
E-mail: jls and wf@ipb.uni-bonn.de

Comm.III Symposium’94, WG/2

3. Manesh Kokare, B.N. Chatterji and P.K. Biswas, “A survey on current content based image retrieval methods,” IETE Journal of Research, Vol. 48, No.3&4, 261-271, May-Aug 2002.
4. B.S. Manjunath and W.Y. Ma, “Tcxture features for browsing and retrieval of image data”, IEEE Trans. on Pattern Analysis and Machine Intelligence, 8(8), 837- 842, Aug 1996.

Thank You